

AAML Final Project - Pruned Wav2Letter

Outline

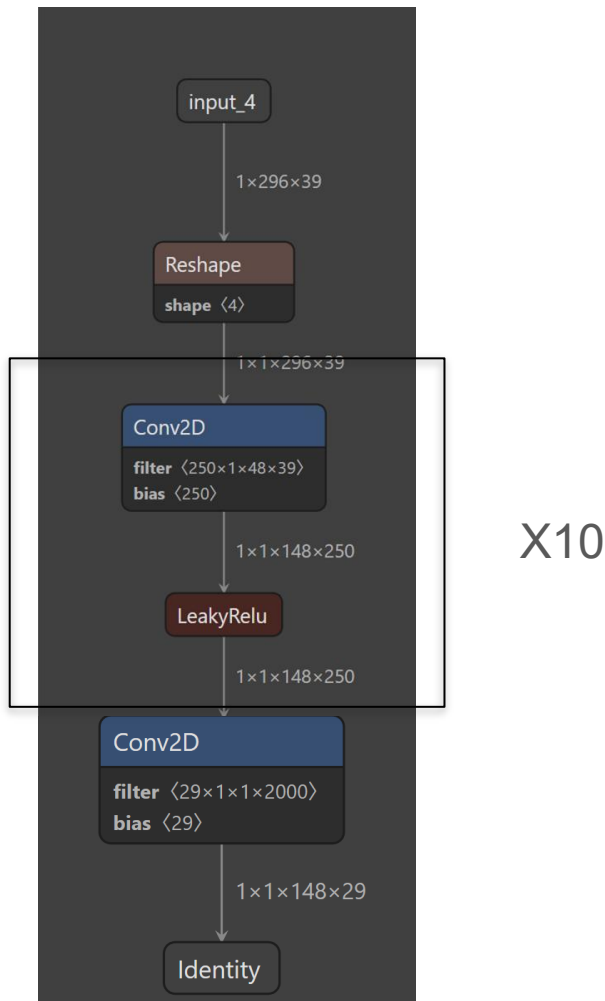
- Challenge
- Methodology
- Evaluation Result
- Conclusion

Outline

- Challenge
- Methodology
- Evaluation Result
- Conclusion

Challenge

- Pruned Wav2Letter



Challenge

- Baseline (Software) Convolution
 - Multi-level for-loops (out_y / out_x / out_ch / filter_y / filter_x / in_ch)
 - Each MAC is surrounded by address computation, boundary checks, and loop-control overhead.

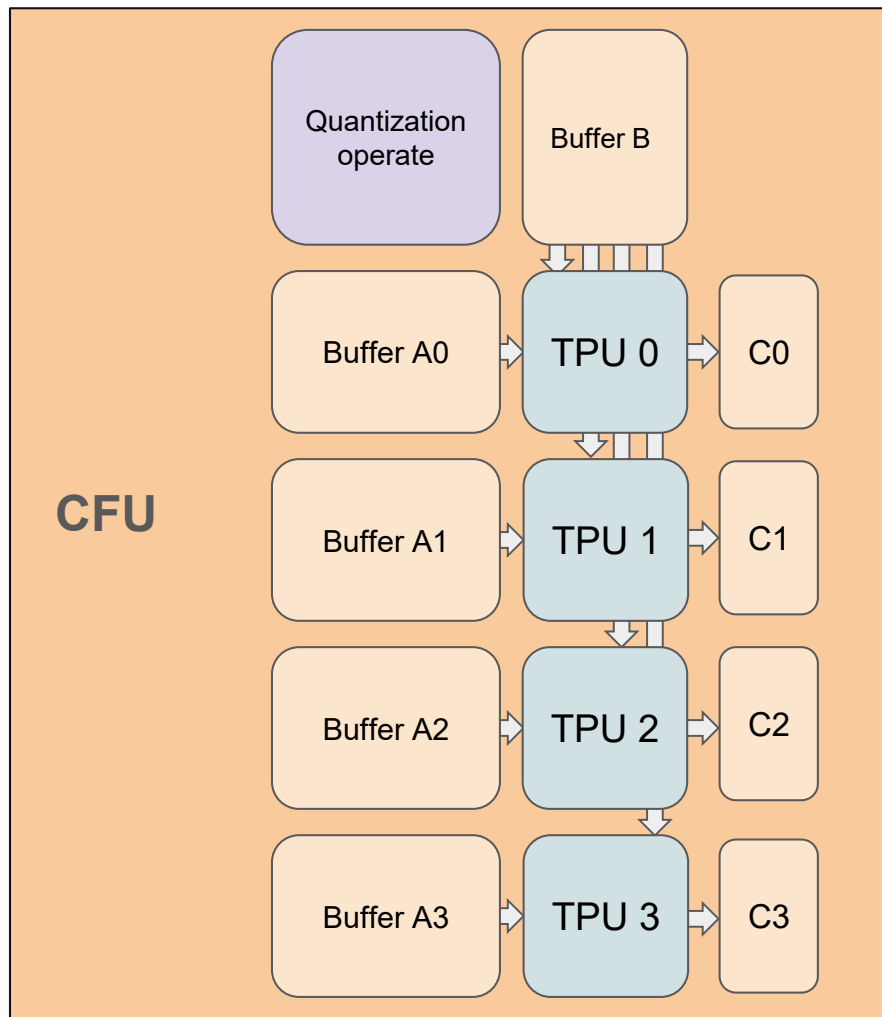
```
acc += filter_val * (input_val + input_offset);  
#MAC≈B·OH·OW·OC·FH·FW·C_in/group
```

Outline

- Challenge
- **Methodology**
- Evaluation Result
- Conclusion

HW Architecture & SW implement

- Quad-Core Data Parallelism
 - Utilizes a SIMD-like architecture with 4 parallel TPU cores driven by software scheduling.
 - Distributes output pixel computation (M-dimension) across cores using QUAD_CORE_STEP to maximize hardware utilization.



ISA (OP code)

- OP_1: **Set Input Offset**
- OP_2 / 7: **Set K, N / M**
- OP_3 / OP_13 / OP_20 / OP_25: **Write Buffer A0–A3**
 - Write packed input to A buffers of each TPU
- OP_4: **Write Global Buffer B**
 - Broadcast to all TPU cores
- OP_10: **TPU Start Computing**
 - Launch 4 TPU cores simultaneously
- OP_5 / OP_15 / OP_22 / OP_27: **Read Buffer C0–C3**
 - Read accumulator results from each TPU
- OP_6 / OP_8: **Set quantization parameters**
- OP_9 / OP_11: **Quantize and activate output**

```
localparam OP_SET_OFFSET = 7'd1;
localparam OP_SET_K_N    = 7'd2;
localparam OP_WRITE_A0   = 7'd3;
localparam OP_WRITE_B    = 7'd4;
localparam OP_READ_C0    = 7'd5;
localparam OP_SET_MS2    = 7'd6;
localparam OP_SET_M      = 7'd7;
localparam OP_SET_MS     = 7'd8;
localparam OP_QTIZE_ACC  = 7'd9;
localparam OP_START      = 7'd10;

localparam OP_RELU       = 7'd11;
localparam OP_BIAS_OFFSET = 7'd12;

localparam OP_WRITE_A1   = 7'd13;
localparam OP_READ_C1    = 7'd15;

localparam OP_WRITE_A2   = 7'd20;
localparam OP_READ_C2    = 7'd22;

localparam OP_WRITE_A3   = 7'd25;
localparam OP_READ_C3    = 7'd27;
```

Conv

```
int m_temp = M_curr_step;
int M_c0 = (max_rows_per_core < m_temp) ? max_rows_per_core : m_temp; m_temp -= M_c0;
int M_c1 = (max_rows_per_core < m_temp) ? max_rows_per_core : m_temp; m_temp -= M_c1;
int M_c2 = (max_rows_per_core < m_temp) ? max_rows_per_core : m_temp; m_temp -= M_c2;
int M_c3 = m_temp;

pack_core_input(0, ..., M_c0, OP_WRITE_A0, k_block_start, k_block_size);
pack_core_input(M_c0, ..., M_c1, OP_WRITE_A1, k_block_start, k_block_size);
pack_core_input(M_c0 + M_c1, ..., M_c2, OP_WRITE_A2, k_block_start, k_block_size);
pack_core_input(M_c0 + M_c1 + M_c2, M_c3, OP_WRITE_A3, k_block_start, k_block_size);
```

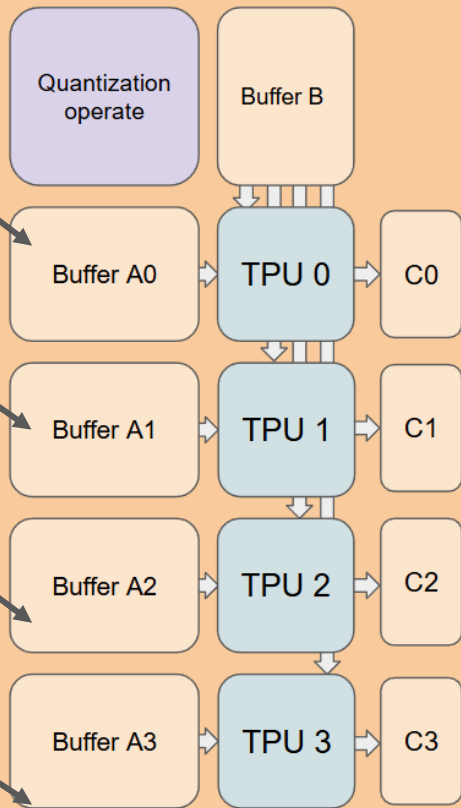
	K_size		
A0	1	2	M_size
A1	3	4	
A2	5	6	
A3	7	8	

OP_3: Write A0 buffer
 $A[0 : M_0-1] \times K$

OP_13: Write A1 buffer
 $A[M_0 : M_0+M_1-1] \times K$

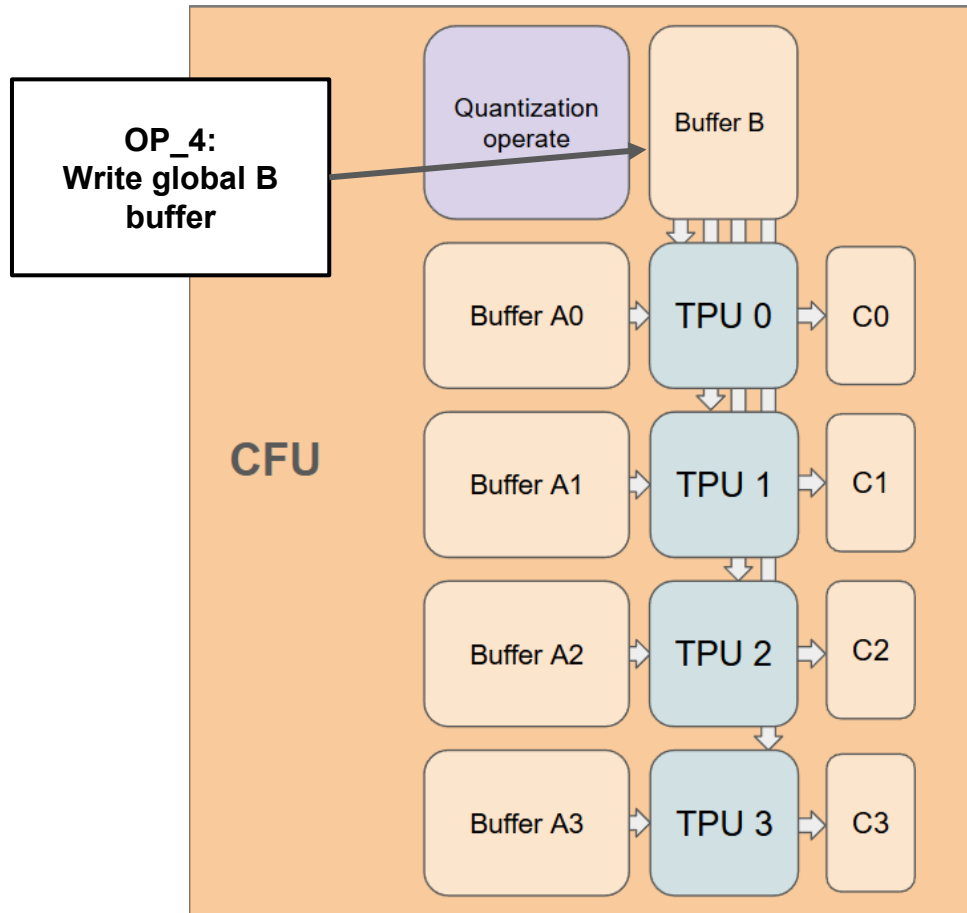
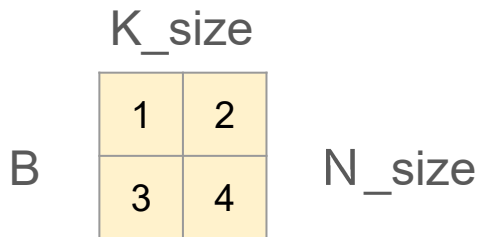
OP_20: Write A2 buffer
 $A[M_0+M_1 : M_0+M_1+M_2-1] \times K$

OP_25: Write A3 buffer
 $A[M_0+M_1+M_2 : M-1] \times K$



Conv

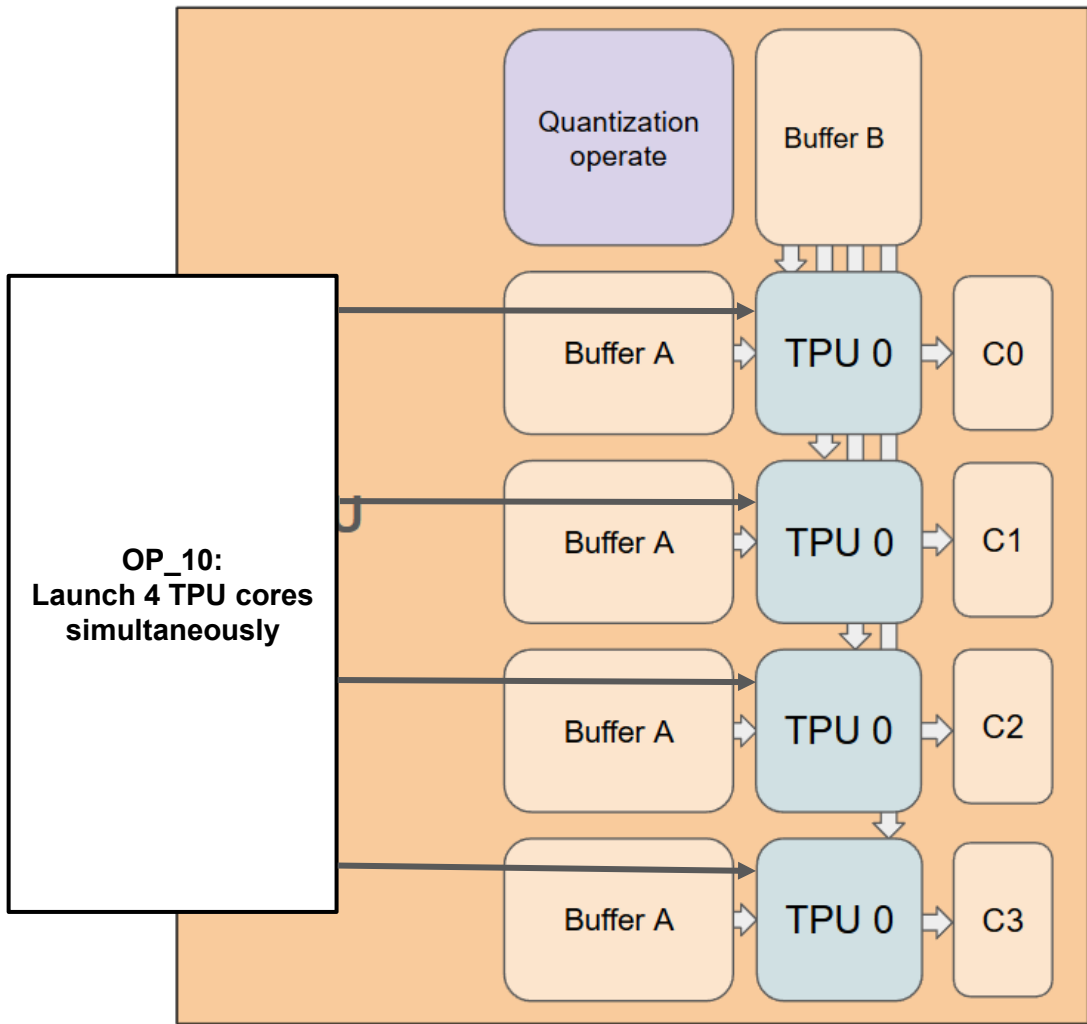
```
int tile_idx = out_ch / 4;
const uint32_t* tile_w_ptr =
    packed_weights_ptr + (tile_idx * words_per_group) + block_word_start;
int w_remain = block_word_size;
int i = 0;
while (w_remain >= 4) {
    cfu_op0(OP_WRITE_B_CAST, i++, *tile_w_ptr++);
    cfu_op0(OP_WRITE_B_CAST, i++, *tile_w_ptr++);
    cfu_op0(OP_WRITE_B_CAST, i++, *tile_w_ptr++);
    cfu_op0(OP_WRITE_B_CAST, i++, *tile_w_ptr++);
    w_remain -= 4;
}
```



Conv

```
cfu_op0(OP_START_RUN, 0, 0);
```

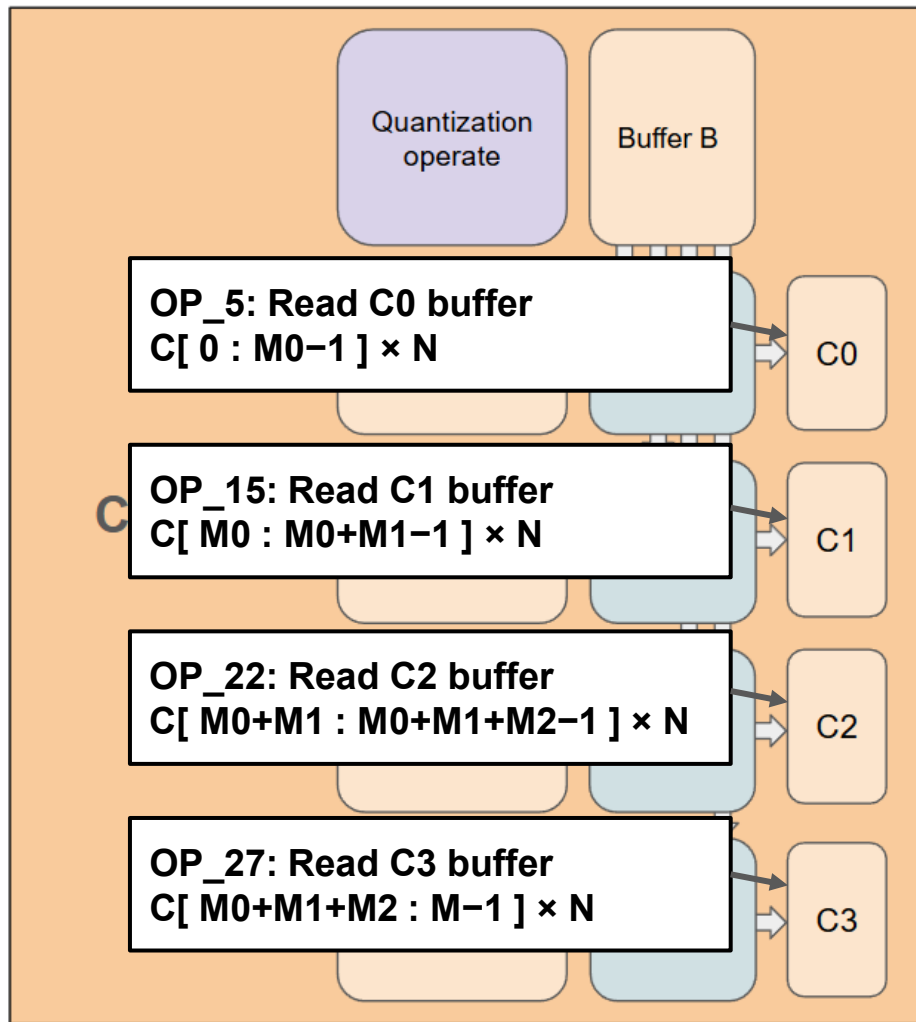
- When input and weight buffers are ready, we issue **OP_START** to launch all four TPU cores simultaneously.



Conv

```
read_core_output(0, ..., M_c0, OP_READ_C0);  
read_core_output(M_c0, ..., M_c1, OP_READ_C1);  
read_core_output(M_c0 + M_c1, ..., M_c2, OP_READ_C2);  
read_core_output(M_c0 + M_c1 + M_c2, M_c3, OP_READ_C3);
```

	N_size		M_size
C0	7	10	
C1	15	22	
C2	23	34	
C3	31	46	

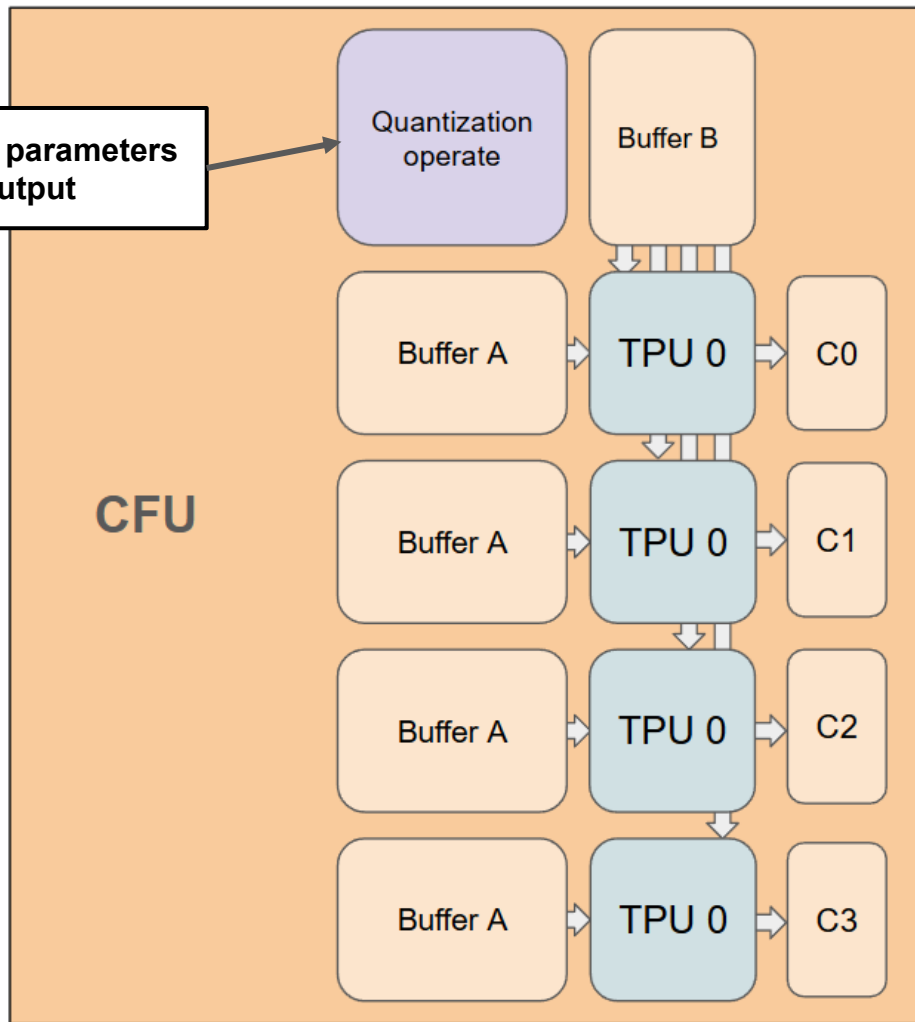


Conv

OP_6 / OP_8: Set quantization parameters
OP_9 / OP_11: Quantize and output

```
acc = cfu_op0(9, acc, output_offset);  
if (acc < act_min) acc = act_min;  
if (acc > act_max) acc = act_max;  
dst_base[m_local * OC + n] = (int8_t)acc;  
}
```

- **acc** is the accumulator value read from the C buffer, which is then processed by bias addition, quantization, activation, and finally written back as the output.



Leaky Relu

```
static const int32_t quantized_min = std::numeric_limits<T>::min();
static const int32_t quantized_max = std::numeric_limits<T>::max();
for (int i = 0; i < flat_size; ++i) {
    const int32_t input_value = input_data[i] - params.input_offset;
    int32_t unclamped_output;
    if (input_value >= 0) {
        unclamped_output = params.output_offset +
            MultiplyByQuantizedMultiplier(
                input_value, params.output_multiplier_identity,
                params.output_shift_identity);
    } else {
        unclamped_output = params.output_offset +
            MultiplyByQuantizedMultiplier(
                input_value, params.output_multiplier_alpha,
                params.output_shift_alpha);
    }
    const T clamped_output =
        std::min(quantized_max, std::max(quantized_min, unclamped_output));
    output_data[i] = static_cast<T>(clamped_output);
}
```



```
const int32_t m = params.output_multiplier_identity;
const int32_t s = params.output_shift_identity;
const int32_t m1 = params.output_multiplier_alpha;
const int32_t s1 = params.output_shift_alpha;
const int32_t poedisonp = params.output_offset;
const int32_t input_offset = params.input_offset;

static const int32_t quantized_min = std::numeric_limits<T>::min();
static const int32_t quantized_max = std::numeric_limits<T>::max();

cfu_op0(2, quantized_min, quantized_max);
cfu_op0(8, m, s);
cfu_op0(6, m1, s1);

for (int i = 0; i < flat_size; ++i) {
    const int32_t input_value = input_data[i] - input_offset;
    output_data[i] = static_cast<T>(cfu_op0(11, input_value, poedisonp));
}
```

- **Left:** Software-only implementation of quantization and activation using **MultiplyByQuantizedMultiplier**, branch on sign, then clamp in software.
- **Right:** Quantization and activation offloaded to CFU hardware.

Outline

- Challenge
- Methodology
- **Evaluation Result**
- Conclusion

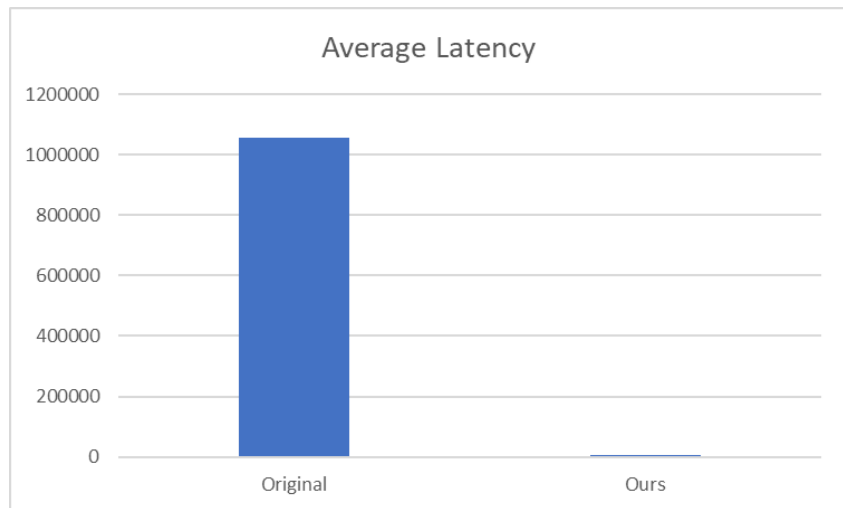
Evaluation Results

Original

```
--- Evaluation Complete ---  
Total Inferences: 5  
Average Latency: 1056573.70 ms  
Accuracy (WER): 72.22%  
(Word Error Rate: 27.78%)
```

Ours

```
--- Evaluation Complete ---  
Total Inferences: 5  
Average Latency: 7565.91 ms  
Accuracy (WER): 72.22%  
(Word Error Rate: 27.78%)
```



Latency speedup = 139.64

Outline

- Challenge
- Methodology
- Evaluation Result
- Conclusion

Conclusion

- The current design is primarily limited by buffer I/O overhead (data loading and unloading).
- Future work could focus on implementing burst mode transfers by holding the CMD signal high, allowing for continuous data streaming to the buffer and significantly reducing handshake overhead.
- It may be necessary to implement a FIFO to enable continuous data streaming into the buffer.

Thanks for listening